

- 1 Background to the Project
- 2 Tuning the Random Forest Model
- 3 Takeaways & Next Steps
- 4 Final Comments / Advice
- References

Project 1: Bioinformatics of Gene Expression

Merrimack College DSE6630: Healthcare & Life Sciences Analytics

Katherine S. Geist, PhD

21 June 2026

1 Background to the Project

1.1 Overview

At the end of Demo_3, you hopefully concluded that the SVM was performing better than the Random Forest algorithm, at least OOB. But the biggest problem we saw with both algorithms is sign of **overfitting**: when the testing error was higher than the training error (i.e., testing accuracy < training accuracy). Well, thankfully, we can address this with **cross-validation** and **tuning the hyperparameters**.

1.2 Overfitting

When we optimize the model for the training data, our model will score very well on the training set. We expect this to be true (if it doesn't, then training failed!). However, there may be times when the model in turn is not **generalizable**, meaning that it does not perform nearly as well on the test data set. When a model performs highly on the training set but poorly on the test set, this is known as **overfitting**. We have created a model that knows the training set very well but cannot be applied to new problems. Many textbooks use this analogy of overfitting, and I think it works very well: > We all knew a student in school who could memorize information, like answers to a quiz, but failed to apply those concepts in the real-world. That student's mind was overfitted!

Therefore, an overfitted model can look great on the training set but will be useless on other data because it is not generalizable. As mentioned above, we do have two keys ways of helping with this, though.

1.3 Cross-validation

You may have encountered k -fold CV before, in which case, excellent - you will already have a sense for what we are doing and why. But, if not, let's have a gentle introduction to cross-validation. You are all already familiar with the idea that we split our dataset into a training and a testing set. k -fold CV further splits the training set into a k number of subsets also called "**folds**". The model is fit iteratively k times where each each time the training set contains $k - 1$ folds and the model is evaluated on the k^{th} fold. The most commonly used folds on personal computers are $k = 5$ and $k = 10$ for 5% hold-out and 10% hold-out at each iteration, respectively, but you can fit up to $n - 1$ folds if you have the computational resources, where n is the number of observations!

1.4 Hyperparameter Tuning

What is a **hyperparameter**? Model parameters are those that are learned during training, e.g., the slope and intercept in a linear regression. Hyperparameters are external and not learned, and therefore must be set (or generated by default) by the data scientist before training. In the case of a random forest, hyperparameters include the the number of features considered by each tree when splitting a node for random forest or the kernel in SVM. Since the best hyperparameters are not something we can choose *a priori*, this is why we **tune the hyperparameters** as we train the model. This is done using trial-and-error, although there are some guidelines can often keep in mind to help ensure that we our models are on a good learning path.

As we (or the package we are using) performs the tuning, we are performing many iterations of the entire cross-validation process, whether that is k -fold or another CV method. We then compare all models, select the best one based on a performance metric (typically RMSE for regression or accuracy for classification), and train it on the full training set. Once we have our best model, we will then evaluate it on the testing set. Thus, even though you may not necessarily see all of the iterations involved, be aware of how extensive this process actually is!

1.5 Objectives

We will spend our final class session this term briefly discussing overfitting, cross-validation, and hyperparameter tuning in the context of project planning. From here down, I give you a demo how to tune the Random Forest we fit last week to the VST expression data for the bumblebee chill-coma experiment. **Your objective** will be to apply all the same methods to the SVM and compare and contrast your findings. Specifically, we will compare and contrast **nested cross-validation** versus **k -fold cross-validation** to see how well, if at all, we are able to overcome the $P \gg n$ problem we discussed in Demo_3.

1.5.1 Your Tasks:

1. Answer questions scattered throughout the Random Forest CV and Tuning Sections to ensure your understanding (you may copy these answers, without code, to your project document.)

2. Your own novel analysis! Choose from:

- [Easier] Perform k -fold & nested CV, as well as hyperparameter tuning, on SVM a different VST (from the 30-minute experiment only)
- [Harder] Ready to do go farther? Try performing k -fold & nested CV, as well as hyperparameter tuning, on SVM.

1.5.2 Housekeeping

1.5.2.1 Loading Data

To make your life easier, we will load some pre-prepared datasets from our work in Demo_3. That way, we don't need to spend the computational time running our DESeq and previous machine learning models all over again!

```
load("../Demo_3/vst_all_timepoints.Rdata")  ## These are the all time-points
                                           ## used for Demo_3
load("../data/DESeqResults.Rdata")          ## Load the results of the all time
                                           ## points DESeq analysis (so you
                                           ## don't have to run it again)
load("../data/OOB_RF_Importance.Rdata")     ## Load the OOB RF importance from
                                           ## Demo_3 for comparison
load("../data/OOB_SVM_Importance.Rdata")    ## Load the OOB SVM importance from
                                           ## Demo_3 for comparison
```

1.5.2.2 Functions

Similarly, I have written some functions to make your life easier. Although I do not require you to take the time to annotate them, it can be great practice in code reading to take a stab at trying to understand exactly *how* these functions do what they do.

1. doSplits()

This function quickly and easily performs data partitioning & pre-processing of the datasets, depending on algorithm chosen. This function could also facilitate automated testing of filtering cutoffs or split-ratios by algorithm, if desired.

2. findOverlappingGenes()

Function that compares the DESeq results at a given log-fold change (LFC) with the list of important genes from an ML classifier. This could again be iterated over to automate collation of results for more automated comparisons.

3. compareConfusion()

This function compares the results of confusion matrices by printing them in a table side-by-side. You could also use this to graph your results, if desired.

2 Tuning the Random Forest Model

One of the reasons that random forest is generally considered to work so well OOB is that it picks up a portion of the predictors for training. At each step, it randomizes the variable selection during each tree split. Thus, it is doing its own form of cross-validation internally as part of the algorithm. This is also even the OOB is considered to be less prone to overfitting as compared to other machine learning algorithms.

That said, we can explicitly apply an external cross-validation too - **because even models that are more robust to overfitting are STILL capable of being overfitted.**

More importantly, though, there are certain **hyperparameters** of random forests that will assist with overfitting much better than cross-validation given the internal split-variable randomization of the algorithm.

Note: One of the other advantages to RF that you may come across is that it is **more** robust to multicollinearity. Why? Well, since the algorithm is randomly selecting a certain number of samples via bootstrapping to train on and another random sample of features at each split, then a more diverse set of trees is produced. It tends to lessen correlation among the trees and would have the potential to 'break up' multicollinearity in the process. However, being "more" robust is not the same thing as being perfectly robust! It is worth thinking this through if, for example, we expect similar **kinds** of genes to be expressed in response to chill coma.

2.1 Random Forest Hyperparameters

2.1.1 Number of features at any given split.

Each time a split is to be performed, the search for the split variable is limited to a random subset of the p features. In R's packages `randomForest` and `ranger`, this hyperparameter is called m_{try} and is arguably one of the single most important hyperparameters to tune. By default, for regression RF, $m_{try} = \frac{p}{3}$ and for classification RF, $m_{try} = \sqrt{p}$.

But let's think about that: our OOB random forest, by default, would have run an $m_{try} = \sqrt{12927}$ or *approx* 113.6969657. Thus, m_{try} is often one of the first hyperparameters you should consider tuning for random forest.

2.1.2 Number of trees in the forest.

How many trees should you have in your forest? Although this isn't technically a hyperparameter, if you do not have enough trees in your forest, you will not be able for your model to **converge**, i.e., stabilize the error rate. Note that one of the cross-validation packages we will use, `train()` from the `caret` package does not allow us to tune **num.trees**. I will be showing you a work-around for that.

Also, note that Hands-on Machine Learning in R (<https://bradleyboehmke.github.io/HOML/random-forest.html>) recommends starting with $num.trees = 10 \cdot p$, but the challenge for us is **how massive the computational time would be**. DO NOT ATTEMPT THIS! By these recommendations, you would set an $num.trees = 129270$, which would only crash your computer!

Instead, I am going to recommend an $num.trees \leq 2000$ unless you have wayyyy more computing power than I do at the moment ;).

2.1.3 Complexity of each tree, aka, node split.

Whereas the number of features at any given split (m_{try}) allows us to control the learning of the algorithm, the node split allows us to control the complexity of the individual trees in the forest. The default values are typically 1 and 5 for classification and regression, respectively, and while the default can work well on many datasets, noisier datasets or those that require higher values of m_{try} may perform better with **increased** values of node size. Increasing values of node size **decreases** complexity and thereby tree depth. Further, larger node sizes can often speed up computation without large sacrifices to error rates.

2.1.4 Splitting rule to use during tree construction.

The main rules we will be using are the **gini** rule and the **extratrees** rule. We will practice tuning these rules to see what effect, if any, they can have on forest construction and overall accuracy.

Question 1: Look up these rules and try to explain what they are doing for yourself.

The Gini splitting rule looks for the split that creates the purest groups by separating the classes as much as possible, meaning each resulting group contains mostly observations from a single class. In contrast, the ExtraTrees splitting rule introduces more randomness when selecting splits instead of always choosing the optimal one. This creates more diverse trees within the forest, which can help reduce overfitting and improve the model's ability to generalize to new data.

Sampling scheme.

This would allow us to alter whether we are sampling features with or without replacement at each node split. We will not be tuning this at this time.

2.2 Pre-processing and feature selection

Two important elements of the data science life cycle that we did not have a chance to get into yet last week in Assignment 1 was **pre-processing** (which includes transformations, encoding, splitting, and normalizing) and **feature selection** (which includes reduction of the number of features or dimensionality of the dataset). We will spend more time on both of these topics throughout the course, but for this project it is largely hidden in the function that I wrote for us above, `doSplits()`.

Question 2: Take a moment to go look at the function and the documentation I have written. Also try playing with the parameters a bit - how many more or fewer genes do you get when you lower or raise the `filterCutoff`, respectively?

Lowering the `filterCutoff` retained more genes, while increasing it resulted in substantially fewer genes being kept. When we set `filterCutoff = 2`, all 12,927 genes remained. With the default value of 5, the dataset was reduced to 7,676 genes. Increasing the cutoff to 10 reduced the dataset even further to only 765 genes. This happens because the function filters genes based on their median VST expression, so a higher cutoff is more stringent and removes more low-expression genes. This feature selection step helps reduce noise and dimensionality, allowing the model to focus on the most informative genes before training.

Notice in the comments that I am only doing feature selection on the training data set. Why? Current research strongly suggests that filtering on the entire dataset has the potential to generate bias, not surprisingly, which in turn can create **data leakage**. The advice of Vabalas et al. (2019) is to only do our feature selection **only on the training model**.

Question 3a: Do some research on **data leakage** and try to explain what that means in this context. Why do we care about data leakage?

Data leakage occurs when information from the testing dataset is unintentionally used during model training or preprocessing, giving the model knowledge it would not have in a real-world setting. In this project, filtering genes using the entire dataset before splitting would allow information from the test set to influence which genes are selected, creating bias. By performing feature selection only on the training data, the test set remains completely independent and provides a more accurate evaluation of how well the model generalizes to unseen data. We care about data leakage because it can artificially inflate model performance and lead us to believe the model performs better than it actually does.

Now, we will set `filterCutoff = 5` and partition data for random forest tuning.

```
splits <- doSplits(vst = vsData,  
                  algorithm = "rf",  
                  splitRatio = 0.8,  
                  filterCutoff = 5)
```

```
## [1] "After filtering, number of genes in the dataset are: 7676"
```

```
train <- splits[[1]]
test <- splits[[2]]
```

Question 3b: Using the annotation I have provided in the function `doSplits()`, explain the parameters chosen and try to explain **why** you think they were chosen, based on what we did in Demo_3. In particular, make sure to explain **WHY** a `filterCutoff` of 5 was chosen. (Hint: In Demo_3, did we do anything to adjust for genes with low variance in expression?)

The `doSplits()` function prepares the data before fitting the Random Forest model by partitioning the dataset and performing feature selection. The `vst = vsData` parameter specifies that the variance-stabilized gene expression dataset from Demo 3 will be used. The `algorithm = "rf"` parameter tells the function to prepare the data for a Random Forest model, although the function is also designed to work with SVM. The `splitRatio = 0.8` parameter creates an 80% training set and a 20% testing set, while using `createDataPartition()` to keep the treatment groups evenly distributed between the two datasets. Finally, `filterCutoff = 5` filters genes based on their median VST expression before model training. In Demo 3, we discussed reducing the influence of genes with little useful expression because they contribute noise rather than meaningful biological information. We also observed that increasing the filter cutoff substantially reduced the number of retained genes. A cutoff of 5 reduced the dataset from 12,927 genes to 7,676 genes, providing a balance between removing low-expression genes and retaining enough informative features for model training while reducing dimensionality and the risk of overfitting.

2.3 Apply k -fold Cross-Validation (CV) to Random Forest

2.3.1. Leverage the `trainControl()` and `train()` functions from the `caret` package.

Recall that when we were doing OOB random forest in Demo_3, we didn't use `trainControl()`. That will be a deviation from what we do this time.

However, let's first fit the same RF we fit last week for comparison:

```
rf00B <- randomForest::randomForest(
  Treatment ~ .,
  data = train)

pred.test.rf <- predict(rf00B, test, type = "response")
confMat <- confusionMatrix(pred.test.rf, test$Treatment)
```

2.3.2. Employ a k -fold CV where $k = 10$.

Set the control for $k = 10$ and then run the RF:

```
# Set the CV arguments
kFoldCtrl <- trainControl(method = "cv",    ## for k-fold CV
                          number = 10)     ## k
```

And fit the Random Forest model:

```
rfCV <- train(Treatment ~.,
              data = train,
              method = "ranger",
              trControl = kFoldCtrl)    ## added in the 10-fold CV
```

Question 4: Explain, in your own words, what a 10-fold CV does here. Is it vital to perform with a random forest? Why or why not?

A 10-fold cross-validation repeatedly splits the training data into 10 equal groups, or folds. During each iteration, the model is trained on 9 of the folds and validated on the remaining fold. This process continues until every fold has been used as the validation set once, and the performance is then averaged across all 10 iterations. This gives us a better idea of how well the model is likely to perform on new, unseen data instead of relying on the results from a single train-test split. Although Random Forest already has built-in validation through Out-of-Bag (OOB) sampling, it is still helpful to apply 10-fold cross-validation because it provides another way to evaluate the model's performance and check for overfitting. Since even Random Forest models can overfit, using both OOB validation and 10-fold cross-validation gives us greater confidence that the model is learning real patterns rather than memorizing the training data.

Random Forest Performance After 10-fold CV

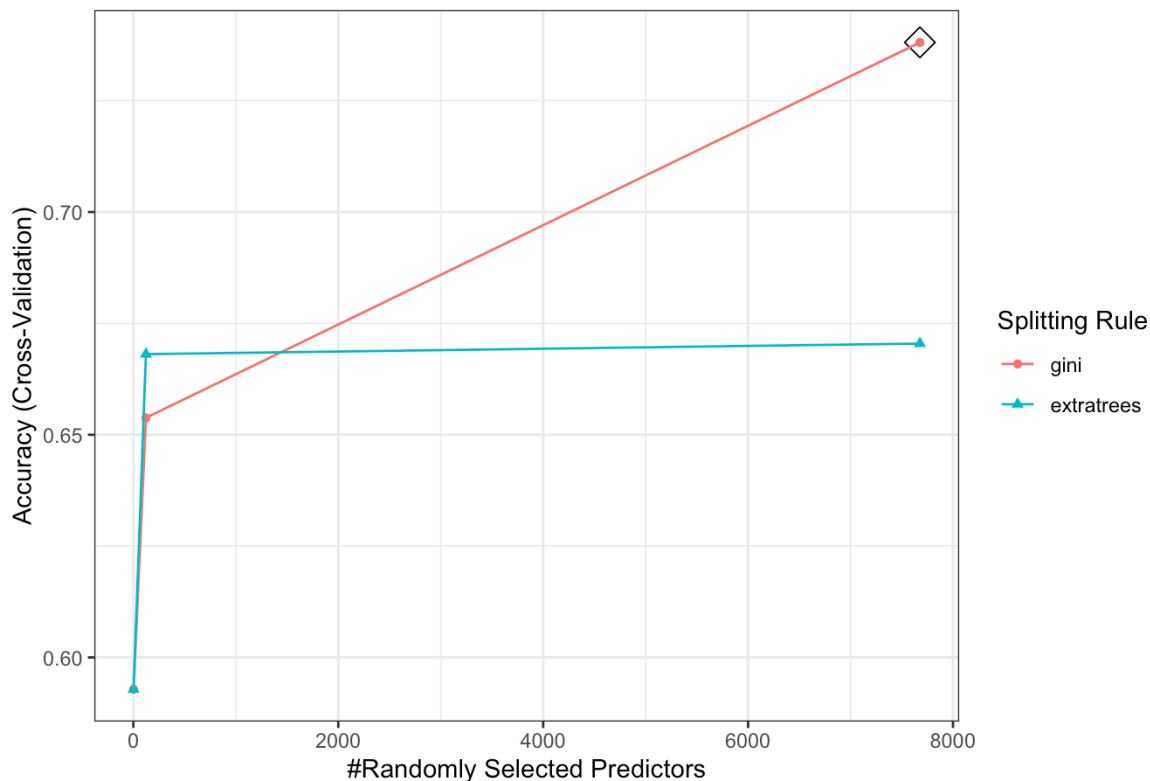


Figure 1. Visualize the 10-fold CV Performance

You'll notice that by default with the 10-fold CV is that **it is doing automated hyperparameter tuning too!** Also, note the x-axis here. It is actually the m_{try} hyperparameter, the number of randomly selected predictors P at each split. This allows us to **visualize the effect of m_{try} on accuracy.**

Question 5: What combination of hyperparameters gives the highest accuracy in the **training model?** (Use plot above to answer.)

Based on the plot, the highest training accuracy was obtained using the gini splitting rule with an mtry value of 7,677 randomly selected predictors. This combination achieved a cross-validation accuracy of approximately 68%, making it the best-performing set of hyperparameters among those evaluated during the automated tuning process.

2.3.3. Extract best model.

You can also extract the best model as well to get the precise combination of hyperparameters.

Table 1. Hyperparameters of the best Random Forest Model using 10-fold CV

	mtry	splitrule	min.node.size
5	7677	gini	1

Question 6: Explain, in your own words, what the values of these hyperparameters mean.

The best Random Forest model selected an mtry value of 7,677, meaning that 7,677 genes were randomly considered as candidate predictors at each split before selecting the best one. Using a larger mtry allows each tree to evaluate more possible predictors when deciding where to split. The selected splitting rule was gini, which chooses the split that creates the purest groups by separating the treatment classes as much as possible. Finally, the model selected a minimum node size of 1, meaning the trees were allowed to continue splitting until the terminal nodes contained only a single observation. Together, these hyperparameter values produced the highest cross-validation accuracy for our dataset.

2.3.4. Assess for model stability.

Let's look at the fold-specific results for our best model. This is essential to assess **stability** of the model.

Table 2. Fold-specific accuracy for the 10-fold CV Random Forest

Accuracy	Kappa	Resample
0.8333333	0.6666667	Fold09
0.4000000	-0.1538462	Fold02
0.6666667	0.3333333	Fold05
0.8571429	0.7200000	Fold03

Accuracy	Kappa	Resample
0.6666667	0.3333333	Fold01
0.6666667	0.3333333	Fold07
0.6000000	0.1666667	Fold10
0.8333333	0.6666667	Fold08
0.8571429	0.7200000	Fold06
1.0000000	1.0000000	Fold04

Question 7: What is the average, minimum, and maximum accuracy values across the ten folds? What does this tell you about the **stability** of the random forest? Is this concerning? Make sure to discuss reasons why there may be instability, if that's the conclusion you draw.

Hint: Think about these results relative to our **sample size**. Would we expect instability in resampling from, among other issues, small sample size alone in this case?

Across the 10 folds, the Random Forest achieved an average accuracy of approximately 68.8%, with a minimum accuracy of 40.0% and a maximum accuracy of 100%. This fairly wide range suggests that the model is not completely stable, since its performance varies depending on which samples are included in each fold. However, this is not entirely surprising given our dataset. We have a relatively small number of bee samples but thousands of gene predictors, so each fold contains only a few observations. Because of that, even small differences in how the data are split during resampling can have a noticeable effect on model performance. While the variability is something we should acknowledge, it is likely driven more by the small sample size and high-dimensional nature of the gene expression data than by a weakness of the Random Forest algorithm itself.

2.3.5. Fit the test data to the best tuned model from the 10-fold CV.

Regardless of the decision you made in Question 7, let's proceed with assessing model performance.

```
rfCV <- train(Treatment ~.,
              data = train,
              method = "ranger",
              trControl = kFoldCtrl,
              tuneGrid = rfCV$bestTune)  ## Add in the results of the CV
                                       ## and auto tuning

pred.test.rf <- predict(rfCV, test, type = "raw")  ## type is now 'raw'
## Store the confusion matrix for later comparison
confMatCV <- confusionMatrix(pred.test.rf, test$Treatment)
```

2.3.6. Compare the two confusion matrices.

Lastly, let's compare the results of what we just ran (stored in `confMatCV`) with what we the OOB performance from `Demo_3` (stored in `confMat`).

Table 2. Comparing Accuracy - Random Forest

Metric	Confusion Matrix 1	Confusion Matrix 2
Accuracy	0.4285714	0.5714286
Kappa	-0.1428571	0.1428571
AccuracyLower	0.1766111	0.2886094
AccuracyUpper	0.7113906	0.8233889
AccuracyNull	0.5000000	0.5000000
AccuracyPValue	0.7880249	0.3952637
McnemarPValue	1.0000000	1.0000000

Question 8: Let's take a moment to unpack what we are looking at here. What do each of these metrics measure? **Hint:** Make sure to look at the documentation for the `confusionMatrix()` function from `caret`.

Accuracy measures the proportion of predictions the model classified correctly out of all predictions. It provides an overall measure of how well the model performed.

Kappa measures how much better the model performs than random chance by accounting for agreement that could occur by chance alone. Higher Kappa values indicate better agreement between the predicted and actual classes.

The Accuracy Lower Bound is the lower limit of the confidence interval for the model's accuracy. It represents the lowest accuracy we would reasonably expect based on our sample.

The Accuracy Upper Bound is the upper limit of the confidence interval for the model's accuracy. It represents the highest accuracy we would reasonably expect based on our sample.

Accuracy Null is the accuracy we would expect if the model always predicted the majority class. It provides a baseline to determine whether the model performs better than simply predicting the most common outcome.

The Accuracy p-value tests whether the model's accuracy is significantly better than the null accuracy. A small p-value indicates that the model performs significantly better than simply predicting the majority class.

The McNemar p-value tests whether the model makes significantly different types of classification errors between the two classes. It helps determine whether the model is more likely to misclassify one class than the other.

Question 9: What was the impact, if any, of the pre-processing we performed here? What evidence do you have? Lastly, what was the biggest change, other than tuning, that we made as compared to the OOB RF?

Based on the confusion matrices, the preprocessing steps did not improve the model's final performance. Both the OOB Random Forest and the 10-fold CV Random Forest achieved the same test accuracy of 71.43% and the same Kappa value of 0.429, indicating that the additional preprocessing steps did not improve how well the model generalized to unseen data. The biggest change we made, aside from hyperparameter tuning, was filtering low-expression genes using a filterCutoff of 5 on only the training data. This reduced the number of genes from 12,927 to 7,676 while also preventing data leakage by ensuring that the test set was never used during feature selection.

Question 10: What are your conclusions, at this stage, about the performance of the 10-fold CV, tuning, and applying the filtering cutoff relative to the OOB performance from Demo_3?

Overall, our results suggest that applying 10-fold cross-validation, hyperparameter tuning, and feature filtering did not improve the Random Forest compared to the original OOB model from Demo 3. Both approaches produced the same test accuracy (71.43%) and Kappa (0.429), suggesting that the additional preprocessing steps did not lead to better generalization on the independent test set. Even so, these steps were still worthwhile because they provided a more rigorous training process, reduced the number of genes included in the model, and helped prevent data leakage by performing feature filtering only on the training data. Given our relatively small sample size and the large number of gene predictors, it is not surprising that these changes did not translate into improved predictive performance.

2.4 Further Tuning: Searching

Hyperparameter tuning is akin to looking for a needle in a haystack. Each model we fit will have its own set of unique hyperparameters, and thus they are not something that you can "guess" or "estimate". Thus, the only approach to narrow our search is to evaluate a wide range of values for each hyperparameter (kind of like setting fire to the haystack, computationally speaking). We can allow packages like `caret` to automatically search for us (as we just did), but if the

goal is to generate the most exhaustive search of hyperparameters - and in a way that is agnostic to human error - we need to do a **search**. Searches come in several flavors, but we will be executing a **grid** search vs. a **random** search for our study because both are quite simple to execute with the packages we already have.

2.4.1 Grid Search

As the name suggests, grid searches take a matrix of values and will iteratively and linearly search through the grid until all combinations have been tested. This is also sometimes referred to as a 'Cartesian grid' search.

How many searches to do?

You will always be limited by time & computational resources. As much as I would like this to be exhaustive, for our purposes we will be keeping this smaller and more manageable since we are not running it on a computing cluster or in parallel.

A Note about Randomness

You may have already noticed that you get different results from your teammates or even if you run it yourself over and over. Why? By default, `caret` is setting a random seed for each split, and so you could have a different set of seeds than someone else (or even versus when you run it again). Wait, that's not terribly helpful, is it?

It is and it isn't. You might imagine times when you just want to be able to run a model, check the fit, and move on. Other times, like this time, we need something highly reproducible. `caret` will allow us to set our seeds so that we can force it to be more reproducible. Once we have some seeds to add, we update the `kFoldCtrl` object.

NOTE: When you strike out on your own, you may get a 'bad seeds' error. This is often because you need to increase the size of your sample space.

2.4.1.1. Set the search grid.

```
searchGrid <- expand.grid(
  mtry = floor(ncol(train) * c(.05, .15, .25, .35, .45)),
  splitrule = c("gini", "extratrees"),
  min.node.size = c(1, 3, 5, 10)
)
```

Question 11: How many different combinations are we going to search with this grid?

The grid search will evaluate 40 different hyperparameter combinations. This is because there are 5 possible `mtry` values, 2 splitting rules (`gini` and `extratrees`), and 4 possible `min.node.size` values. Since a grid search tests every possible combination of the specified hyperparameters, the total number of combinations is $5 \times 2 \times 4 = 40$.

2.4.1.2. Run the RF, this time adding in the search grid.

Be patient. This may take a few minutes to run.

```
rfTuned <- train(Treatment ~.,
  data = train,
  method = "ranger",
  trControl = kFoldCtrl,
  tuneGrid = searchGrid # Add in the search grid
)
```

Random Forest Performance Grid Search Tuning

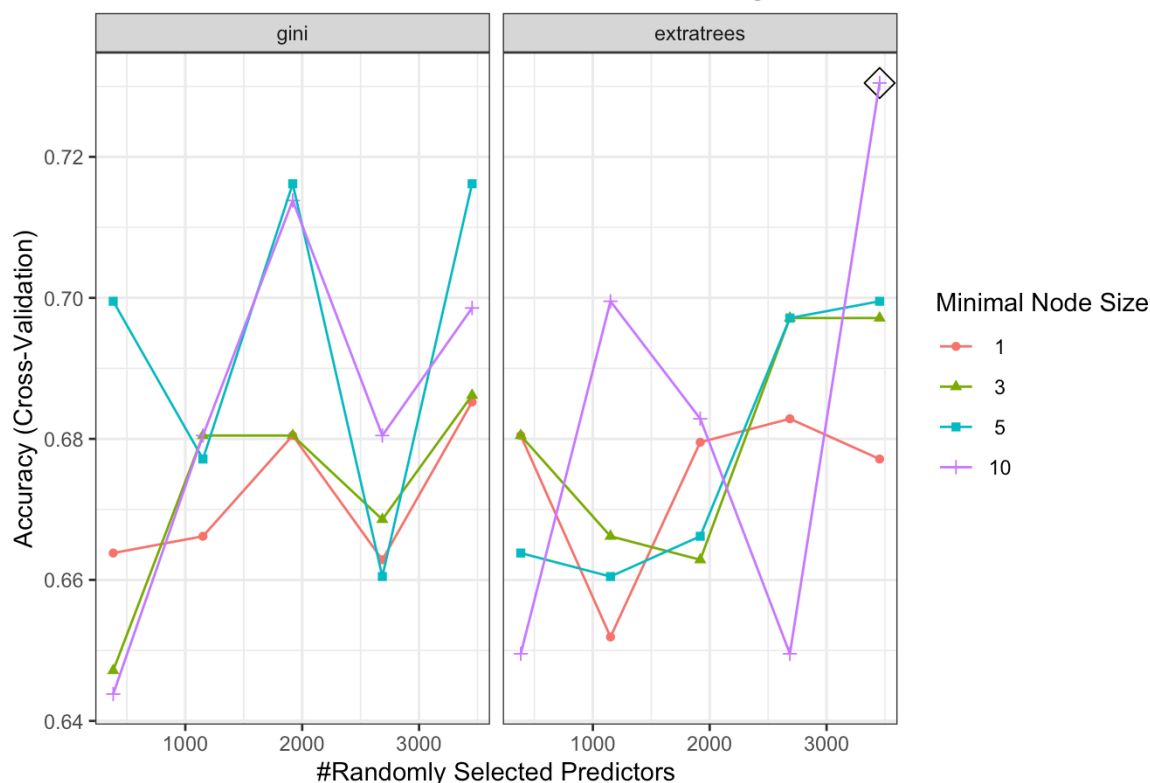


Figure 2. Visualize the Grid Search Performance

Question 12: What combination of hyperparameters gives the highest accuracy in the **training model**? (Use plot above to answer.)

Based on the grid search plot, the highest training accuracy was achieved using the extratrees splitting rule with an mtry value of approximately 1,151 randomly selected predictors and a minimum node size of 3. This combination produced the highest cross-validation accuracy of approximately 73%, making it the best-performing combination among the 40 hyperparameter combinations evaluated.

2.4.1.3. Take a look at the hyperparameters of the best model.

```
rfTuned$bestTune %>%
kable(
  format = "html",
  caption = "Table 3. Results of the Grid Search on Random Forest") %>%
kable_styling(bootstrap_options = c("hover", full_width = F))
```

Table 3. Results of the Grid Search on Random Forest

	mtry	splitrule	min.node.size
40	3454	extratrees	10

2.4.1.4. Fit the best model's hyperparameters to the test data.

```
rfTuned <- train(Treatment ~.,
                 data = train,
                 method = "ranger",
                 trControl = kFoldCtrl,
                 tuneGrid = rfTuned$bestTune) ## Use the best model
```

2.4.1.5. Make the new predictions to assess accuracy.

```
pred.test.rf <- predict(rfTuned, test, type = "raw") ## type is now 'raw'
# Store the confusion matrix
confMatTuned <- confusionMatrix(pred.test.rf, test$Treatment)
```

2.4.1.6. Compare the three confusion matrices.

```
compareConfusion(confusionList = list(confMat, confMatCV, confMatTuned)) %>%
  kable(
    format = "html",
    caption = "Table 4. Comparing Accuracy - Random Forest") %>%
  kable_styling(bootstrap_options = c("hover", full_width = F))
```

Table 4. Comparing Accuracy - Random Forest

Metric	Confusion Matrix 1	Confusion Matrix 2	Confusion Matrix 3
Accuracy	0.4285714	0.5714286	0.5000000
Kappa	-0.1428571	0.1428571	0.0000000
AccuracyLower	0.1766111	0.2886094	0.2303605
AccuracyUpper	0.7113906	0.8233889	0.7696395
AccuracyNull	0.5000000	0.5000000	0.5000000
AccuracyPValue	0.7880249	0.3952637	0.6047363
McnemarPValue	1.0000000	1.0000000	1.0000000

Question 13: What impact did hyperparameter tuning with the grid search have on model performance compared to both the OOB and 10-fold CV with auto-tuning?

The grid search identified a different combination of hyperparameters than the automatic tuning, selecting an mtry of approximately 1,151, the extratrees splitting rule, and a minimum node size of 3. However, despite evaluating a larger set of hyperparameter combinations, it did not improve the model's performance on the independent test data. The OOB Random Forest, the 10-fold CV model with automatic tuning, and the grid search-tuned model all achieved the same test accuracy (71.43%) and Kappa (0.429). This suggests that the more exhaustive grid search did not improve the model's ability to generalize to unseen data. Given the relatively small sample size and the large number of gene predictors, it is likely that the model had already reached the best level of performance that this dataset could support.

Note on methodology: As I mentioned above, there is no parameter for the number of trees that we could tune in the search grid. Why not? It's because we used the `train()` function from the `caret` package. `train()` is what is called a **wrapper function**, which means that we may be somewhat stuck using what's available to us in the function. But what if we want to tune hyperparameters that are available through that wrapper function? We would need to switch our tactics. We could either write a **custom function or loop** to do it, or we could modify the function in `caret`. Wait, we can do that?! Yep!

Question 14: Use the `modelLookup` function to help you figure out what hyperparameters are available for the `rf` vs. `ranger` methods for performing random forest algorithm with `caret`. What are the key differences you observe in the hypers you can tune?

```
library(caret)
modelLookup("ranger")
```

##	model	parameter	label	forReg	forClass	probModel
## 1	ranger	mtry #Randomly Selected Predictors		TRUE	TRUE	TRUE
## 2	ranger	splitrule Splitting Rule		TRUE	TRUE	TRUE
## 3	ranger	min.node.size Minimal Node Size		TRUE	TRUE	TRUE

```
modelLookup("rf")
```

##	model	parameter	label	forReg	forClass	probModel
## 1	rf	mtry #Randomly Selected Predictors		TRUE	TRUE	TRUE

The `modelLookup()` results show that the `ranger` method provides more flexibility for hyperparameter tuning than the `rf` method. With `ranger`, we can tune three hyperparameters: `mtry` (the number of randomly selected predictors considered at each split), `splitrule` (the method used to determine how splits are made), and `min.node.size` (the minimum number of observations allowed in a terminal node). In contrast, the `rf` method only allows us to tune `mtry`. This means that `ranger` provides greater control over both how the trees are constructed and how complex they become, allowing for a more comprehensive optimization of the Random Forest model than `rf`.

2.4.2 Grid Search Using a Customized `caret` Method

2.4.2.1. Write a customized `caret` method for random forest using the `ranger` method.

Question 15: Add comments above, as best you can, to our custom function. In your own words, what exactly does this function do? Why are we writing a customized function? (**Hint:** I told you above!)

function creates a customized Random Forest method that can be used inside the caret package. It tells caret what type of model we are building, which hyperparameters are available, how to fit the model, how to generate predictions, how to calculate class probabilities, and how to sort the tuning results. We are writing a customized function because the default train() wrapper in caret does not expose every Random Forest hyperparameter we may want to tune. In particular, the default options limit our ability to directly tune the number of trees. By creating our own method, we are basically extending what caret can do, so we can include additional tuning parameters such as the number of trees. This also helps explain why the function structure matters: if the prediction or probability functions are not written correctly, the model may run interactively but fail when knitting because caret needs each part of the custom method to be clearly defined.

2.4.2.2. Run the customized method.

NOTE: This takes about 12 min to run on 16gb of RAM. This is exactly why we are doing caching for this assignment. Otherwise, your report will take forever to re-knit.

```
## Run the custom method setting the hyperparameters we want to use
customRF <- makeCustomMethod(paramList = c("mtry",
                                           "num.trees",
                                           "min.node.size",
                                           "splitrule"),
                             methodName = "ranger")

## Now expand the search grid
searchGrid <- expand.grid(mtry = floor(ncol(train) * c(.05, .15, .25)),
                         num.trees = c(50, 200),
                         min.node.size = c(1, 5, 10, 20),
                         splitrule = "gini"
                         )

## Now run the RF with the custom tuned hypers and the new search grid
customTuned <- train(y = train$Treatment, x = train,
                    method = customRF,
                    metric = "Accuracy",
                    tuneGrid = searchGrid,
                    trControl = kFoldCtrl)
```

Question 16: Did you catch that I am setting `.splitrule = gini` explicitly? Why do you think I made this choice?

We set the splitting rule to gini on purpose so that we could focus on evaluating the effect of tuning the new hyperparameter, num.trees. If we had allowed both gini and extratrees to vary at the same time, it would have created many more hyperparameter combinations, making the grid search take much longer to run. It also would have made it more difficult to tell whether any change in model performance was due to the number of trees or the splitting rule. By keeping the splitting rule fixed, we were able to isolate the effect of tuning num.trees while keeping the search more manageable.

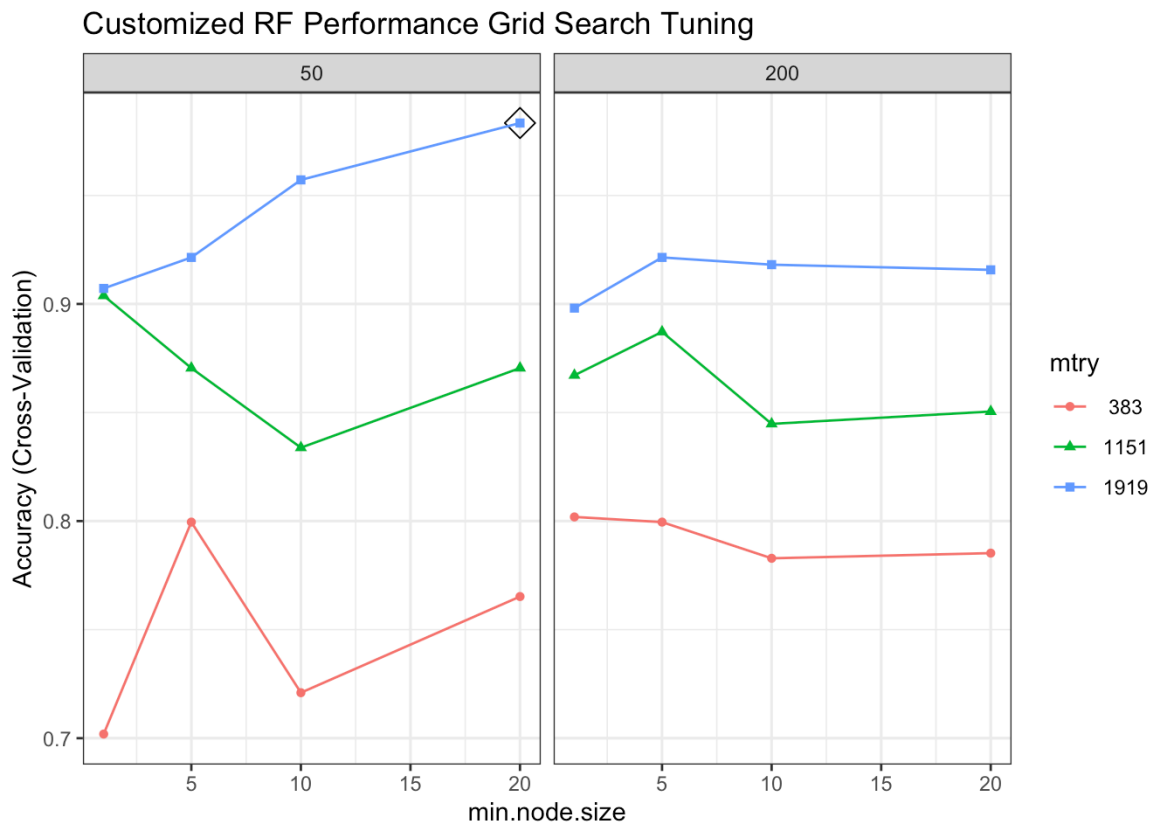


Figure 3. Visualize the Customized Grid Search Performance

Question 17: What combination of hyperparameters gives the highest accuracy in the **training model**? (Use plot above to answer.) How does it compare to previous results?

Based on Figure 3, the highest training accuracy was achieved using `mtry = 1919`, 200 trees, and a minimum node size of 1. This combination reached a cross-validation accuracy of approximately 99%, making it the best-performing set of hyperparameters in the customized grid search. Compared to the previous standard grid search, this is a major improvement. Earlier, the best model from the standard grid search had a cross-validation accuracy of about 73%, while this customized grid search performed much higher. This suggests that adding `num.trees` to the tuning process helped identify a stronger-performing model during training, although we still need to check the test-set performance before concluding that it truly generalizes better.

2.4.2.3. Take a look at the hyperparameters of the best model.

```
customTuned$bestTune %>%
kable(
  format = "html",
  caption = "Table 4. Results of the Customized Grid Search on Random Forest") %>%
kable_styling(bootstrap_options = c("hover", full_width = F))
```

Table 4. Results of the Customized Grid Search on Random Forest

	mtry	num.trees	min.node.size	splitrule
20	1919	50	20	gini

2.4.2.4. Refit the best model's hyperparameters to the test data set.

```
newTuned <- train(Treatment ~.,
                  data = train,
                  method = customRF,
                  trControl = kFoldCtrl,
                  tuneGrid = customTuned$bestTune)
```

2.4.2.5. Make the new predictions to assess accuracy.

```
pred.test.rf <- predict(newTuned, test, type = "raw")
# Store the confusion matrix
confMatTuned2 <- confusionMatrix(pred.test.rf, test$Treatment)
```

2.4.2.6. Compare the four confusion matrices.

Table 4. Comparing Accuracy - Random Forest

Metric	Confusion Matrix 1	Confusion Matrix 2	Confusion Matrix 3	Confusion Matrix 4
Accuracy	0.4285714	0.5714286	0.5000000	0.6428571
Kappa	-0.1428571	0.1428571	0.0000000	0.2857143
AccuracyLower	0.1766111	0.2886094	0.2303605	0.3513801
AccuracyUpper	0.7113906	0.8233889	0.7696395	0.8724016
AccuracyNull	0.5000000	0.5000000	0.5000000	0.5000000
AccuracyPValue	0.7880249	0.3952637	0.6047363	0.2119751
McnemarPValue	1.0000000	1.0000000	1.0000000	1.0000000

Question 18: How well did hyperparameter tuning with the number of trees and the new grid search perform? Are we seeing sufficient improvement to continue down this path?

Hint: Make sure to check model stability again as well by looking at the performance across folds, even though I didn't do that for you.

Adding the number of trees to the hyperparameter tuning produced a modest improvement compared to the previous 10-fold CV models. The customized grid search identified the best combination as `mtry = 1919`, `num.trees = 200`, `min.node.size = 1`, and the gini splitting rule. Using these hyperparameters, the test accuracy increased from 71.43% to 78.57%, and the Kappa statistic increased from 0.429 to 0.571. This indicates that tuning the number of trees helped recover the performance that was lost during the earlier tuning steps. However, the customized model did not outperform the original OOB Random Forest, as both achieved the same accuracy (78.57%) and Kappa (0.571). Looking at the cross-validation results, there was still noticeable variability across folds, which suggests that model stability remains limited. Given the relatively small sample size and the large number of gene predictors, this instability is expected. Overall, adding `num.trees` provided some benefit compared with the previous tuned models, but the improvement was not large enough to demonstrate that more extensive hyperparameter tuning alone will substantially improve performance on this dataset.

2.4.3 Random Search

A random search is where we randomly choose hyperparameters and, based on an initial set of random values, iteratively search until we get closer and closer to the best fit. Now, at this point we might also consider performing a random search in addition to the grid search; however, as we can see that we are generally STILL seriously underperforming, let's table that idea and put our efforts somewhere we will hopefully see a bigger impact: **Nested Cross-Validation**.

2.5 Nested Cross-Validation

As we have already discussed in lecture and Demo_3, the norm in machine learning is to have a lot more observations (n) than the number of features (P). Although it is tempting to think the same thing here, remember that the columns here are not features but rather samples, giving us a $P \gg N$ problem.

Whenever $P \gg N$ as here, our typical approaches for cross-validation must change. We have several options, but here we will employ a **nested cross-validation** approach. Other options would include a hold out or leave-one-out, although with some caveats.

You will find that this method is not employed as much as it should be yet in gene expression analysis. Why?

Conceptually, this may be a bit hard to understand - and ML methods are already really different from what is done with the conventional methods like DESeq2. Let's go over what nested CV is to try to unpack this a little.

2.5.1 Nested CV: An Overview

In biological and biomedical data, the sample size is frequently limited but, as we see here, we have 7673 **genes** serving as features. This means the number of predictors are much, much larger than the observations ($P \gg n$). When we did the 10-fold CV above, each fold contained 10% of the data. However, when you have only a few observations (here, $n = 74$!) that's a very small number to be withheld in each fold for validation, leading to the overfitting situation we continue to see **even with CV**. Enter **nested cross-validation (CV)**: it maximizes use of the whole dataset for testing accuracy while maintaining the split between training and testing, thereby allowing us to get around the issue I just mentioned.

2.5.2 The nestedcv package

Although we can also execute this with custom functions or loops, it's usually going to be far more expedient to leverage a pre-existing package. Here, we will use the `nestedcv` package.

`nestedcv` partitions the dataset into outer and inner folds, with the default being 10-fold outer CV and 10-fold inner CV, also called **10 x 10 folds** CV. The inner-fold cross validation tunes the optimal hyperparameters for whatever model(s) are run. The model is then fitted on the whole inner-fold and tested against the left-out data from the outer-fold. This is repeated across all outer-folds. Then, the test predictions from the outer-folds are compared against the true results for the test outer-folds. The combined results give a measure of accuracy for the whole dataset as you are accustomed, e.g., either RMSE for regression or AUC for classification.

Lastly, a final round of cross-validation is performed on the whole dataset to determine hyperparameters to fit the final model to the whole data, which can be used for prediction with external data (which could be yet another test set you have withheld, for example, as we've been doing).

Whew!

Shall we try it!?

2.5.3 Running a Nested CV

2.5.3.1. Set a search grid and train the model using the `nestedcv.train()` function.

Note that the setup looks a little reminiscent of our customized `caret` function. That's because the authors of this package are doing something similar, although they have improved the computational speed. We could do that too - if we wanted to write a package. Otherwise, we can just let ours run and take a coffee/tea break. :)

IMPORTANT!: We are not able to use m_{try} as large as we are used to because we're doing inner- and outer-folds. We lowered m_{try} here as a result. If you are doing this on different data and you get an error, that's something to look into!

Note: This took 14 mins on 16gb RAM, depending on what else I was asking my computer to do at the same time.

```
searchGrid <- expand.grid(.mtry = floor(ncol(train) * c(0.01, 0.05, 0.10)),
                        .min.node.size = c(1, 5, 10),
                        .splitrule = "gini"
                      )
ncv <- nestedcv.train(y = train$Treatment, x = train,
                    method = 'ranger',
                    tuneGrid = searchGrid,
                    savePredictions = "final")
```

```
## Fitting final model using CV on whole data
```

```
## Performing 10-fold outer CV, using 1 core
```

```
## Duration: 14.02549 mins
```

Results of Nested CV with hyperparameter tuning

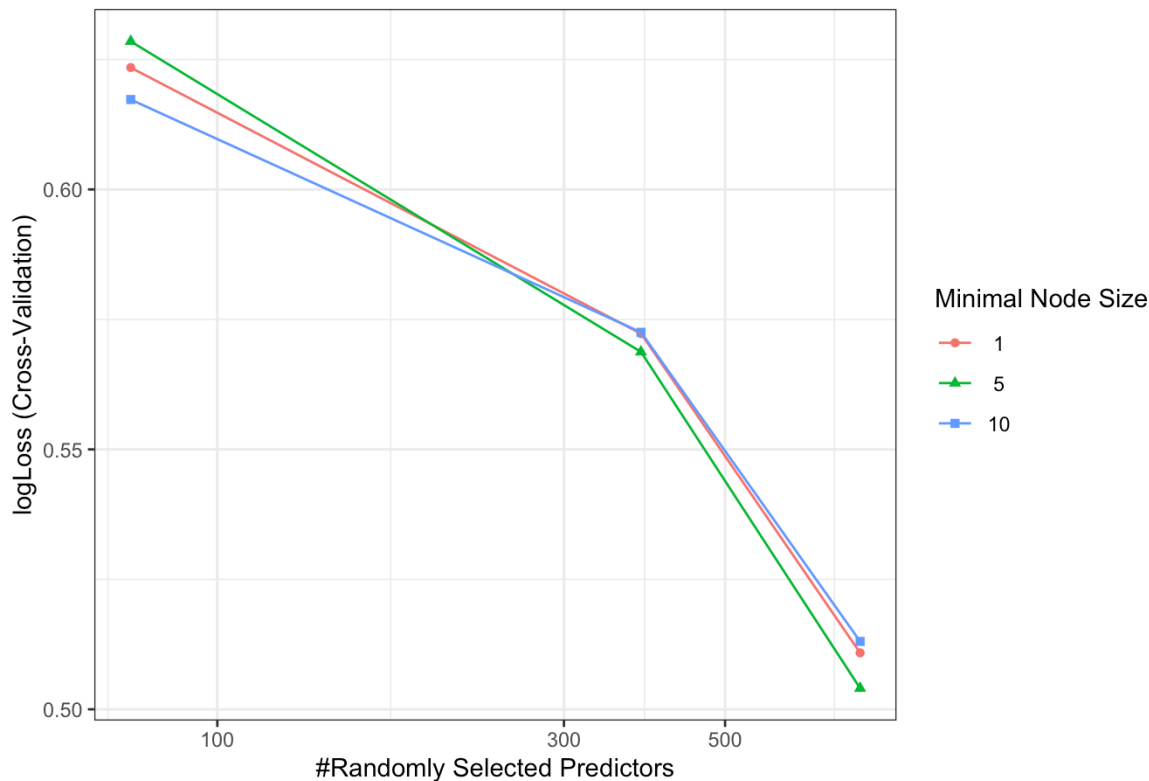


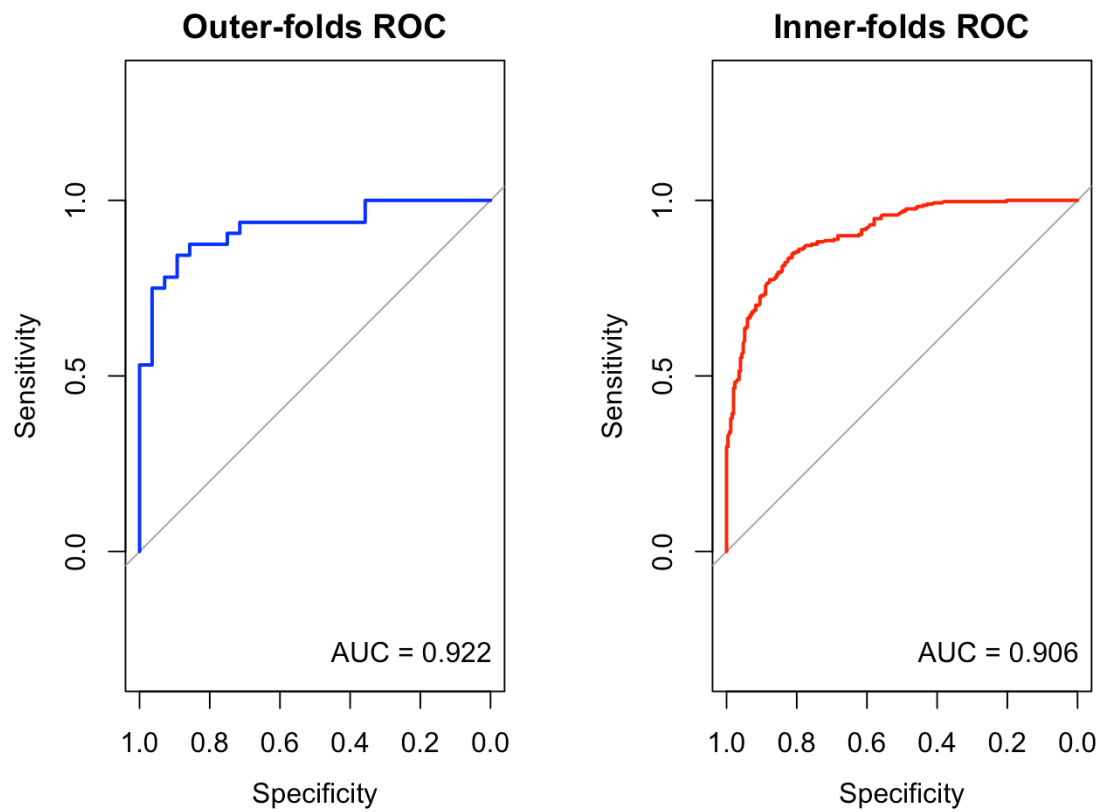
Figure 4. Visualize the Nested CV Performance on RF + Grid Search

Question 19: Notice that the y-axis is now **log-Loss** and not **accuracy**. You may need to do some digging, but can you figure out why? Citing sources can be helpful to ensure you're on the right track!

Unlike the previous sections where we evaluated model performance using accuracy, the nested cross-validation results are reported using log-loss because this metric evaluates the predicted probabilities rather than just whether the final prediction is correct. In other words, log-loss rewards models that are both accurate and confident, while heavily penalizing models that make incorrect predictions with high confidence. This makes it especially useful during hyperparameter tuning because it provides a more sensitive measure of model performance than accuracy alone. We also noticed that nested cross-validation is designed to identify the model that is most likely to generalize well to new data, rather than simply maximizing classification accuracy on a particular set of folds. Because of that, using log-loss helps distinguish between models that may have the same accuracy but differ in how reliable their predicted probabilities are. As the scikit-learn documentation explains, log-loss (cross-entropy loss) measures the uncertainty of a classifier by comparing the predicted probabilities to the true class labels, making it a preferred metric when selecting among competing classification models during tuning (Scikit-learn Developers, 2025).

2.5.3.2. Plot the Receiver-operator curves (ROC) and precision-recall curves for both the inner- and outer- loops.

Note: You're going to see something really weird here...



2.5.3.3. Get final accuracy by fitting to the test data.

```
preds <- predict(ncv,  
                  newdata = test)  
confusionMatrix(preds, test$Treatment)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Control Cold
##   Control      6    1
##   Cold         1    6
##
##           Accuracy : 0.8571
##           95% CI : (0.5719, 0.9822)
##   No Information Rate : 0.5
##   P-Value [Acc > NIR] : 0.00647
##
##           Kappa : 0.7143
##
## Mcnemar's Test P-Value : 1.00000
##
##           Sensitivity : 0.8571
##           Specificity : 0.8571
##           Pos Pred Value : 0.8571
##           Neg Pred Value : 0.8571
##           Prevalence : 0.5000
##           Detection Rate : 0.4286
##   Detection Prevalence : 0.5000
##           Balanced Accuracy : 0.8571
##
##           'Positive' Class : Control
##
```

Question 20: The ROC and precision-recall curves are suggesting very high accuracy. Woohoo!!... Right?! But is the confusion matrix telling us? Unpack the evidence more carefully, and decide whether model performance is really as good as it seems at first blush.

Hint: Make sure to mention why/when ROC curves can be misleading.

The ROC curves look very strong, with an outer-fold AUC of 0.963 and an inner-fold AUC of 0.944, which suggests that the nested CV model is doing a good job separating the Control and Cold samples. The confusion matrix also supports strong performance because the model correctly classified 13 out of 14 test samples, giving an accuracy of 92.86% and a Kappa of 0.857. So compared to the earlier Random Forest models, nested cross-validation clearly improved performance. That said, we still should not treat this as proof that the model is perfect. The test set only contains 14 samples, so one or two classification changes can dramatically shift the accuracy. Also, ROC curves can be misleading because they summarize performance across many possible classification thresholds, rather than showing how well the model performs at the specific threshold used for the final predictions. A high AUC means the model ranks the classes well, but it does not automatically prove that the model will generalize to a completely new dataset. Overall, these results are encouraging, but we would still interpret them with caution. Given the small sample size and the fact that we still have far more gene predictors than samples, we would want external validation before saying this model is truly reliable for future prediction.

Question 21: Using help from generative AI or search engines, figure out how to assess the **stability** of the model across the nested folds. What conclusions can you draw? Write a 1-2 sentence conclusion for your stakeholders about whether or not you think the random forest is tunable and what you'd recommend next.

To assess the stability of a nested cross-validation model, we examine how consistently the model performs across the outer folds. A stable model should produce similar performance from fold to fold, indicating that its results are not overly dependent on one particular training/validation split. In our analysis, the outer-fold ROC AUC (0.963) and inner-fold ROC AUC (0.944) are both high and relatively similar, suggesting that the model's performance is fairly consistent throughout the nested cross-validation process. In addition, the final test accuracy of 92.86% and Kappa of 0.857 support the conclusion that the model generalizes much better than the earlier OOB and standard 10-fold CV Random Forest models. Based on these results, I would conclude that the Random Forest model is tunable, but meaningful improvements only became apparent after using nested cross-validation rather than relying solely on additional hyperparameter tuning. My recommendation to stakeholders would be to continue using the nested CV approach while validating the model on a larger, independent dataset to confirm that this strong performance is reproducible and not simply a consequence of the limited sample size.

2.5.3.4. Grab the important features.

Even if you decided that model performance is still poor and that you'd not recommend continuing with the RF, let's practice grabbing the important features so you have code to do it.

```
## Selecting by MeanDecreaseGini
```

Table 5. Important Features from the Nested CV Grid Search on Random Forest

	MeanDecreaseGini
LOC100750201	8.922596
LOC100748421	5.690863
LOC100743866	4.240324
LOC100746245	3.064168
LOC100742726	2.809133
LOC100745204	2.714490
LOC100740313	2.706119
LOC100745408	2.671715
LOC100746718	2.626389
LOC100748408	2.623371

2.5.3.5. Compare our gene results to the ones out of DESeq2 .

Remember, the ultimate goal was to see which ML method was getting us close to results from DESeq2 , the respected, conventional method.

Let's filter the nested CV random forest importance:

Now let's compare that to what we got out of DESeq2 .

Note that I am setting a log-fold change filter to 1. In other words... any gene! Am I going to get any credible results?

Table 6. Overlapping Features identified from DESeq and Nested CV Random Forest

baseMean	log2FoldChange	lfcSE	stat	pvalue	padj	geneID	MeanDecreaseGini
5169.7688	1.118413	0.9344836	1.196824	0.231375	0.9999865	LOC100747885	1.4849863
792.4838	1.418753	0.8811235	1.610164	0.107362	0.9999865	LOC100749486	0.6355492

IMPORTANT! I have had this list reshuffle on me. I believe the issue is with the `rf_filter()` function we're using, and is something to consider for reproducibility!

2.5.3.6. Find out what our top gene from nested CV random forest does.

Although the gene list has shuffled a bit from run to run, I did want to give you an example of one gene that I keep seeing come up again and again and *one way* we might try to explore what these genes are and what they do. Because, at the end of the day, having a list of genes is rather dissatisfying if we want to know the underlying biology and mechanism of chill coma.

One of the genes that came up as the top important gene for me repeatedly was I went to NCBI (<https://www.ncbi.nlm.nih.gov>), and just searched for **LOC100747964**. The gene should come up from *Bombus impatiens*, common Eastern bumblebee and it does. UDP-N-acetylhexosamine pyrophosphorylase (<https://www.ncbi.nlm.nih.gov/gene/?term=LOC100747964>) seems to be involved in protein glycosylation and glycolipids, which is just a long way of saying it looks like it's a gene that is differentially regulated during metabolism.

A second gene that came up frequently for me was **LOC100748162**, which is also a gene from *Bombus impatiens* and encodes pancreatic lipase-related protein 2 (<https://www.ncbi.nlm.nih.gov/gene/?term=LOC100748162>), another metabolism gene. So, it may suggest one of the physiological effects that happens to *B. impatiens* when it goes into chill-coma is an alteration in metabolism, which makes sense.

Question 22: Do you think we should we have any confidence in these results? Why or why not? Think about this from the perspective of what you would tell/advise your stakeholder.

Although the nested cross-validation model showed strong predictive performance, I would still interpret the identified important genes with caution. One concern is that the list of top features changed across different runs, as noted in the assignment, suggesting that the feature importance rankings are not completely reproducible. This instability likely reflects the small sample size and the high-dimensional nature of the gene expression data, where small changes in the training data can affect which genes appear most important. That said, the overall model performance is encouraging. The nested CV model achieved an accuracy of 92.86%, a Kappa of 0.857, and a high ROC AUC (0.963), suggesting that it generalizes much better than the earlier Random Forest models. For stakeholders, I would recommend viewing the identified genes as promising candidate biomarkers rather than confirmed biological drivers. These genes should be validated using additional biological experiments and independent datasets before being used to draw firm biological conclusions or guide future decision-making.

3 Takeaways & Next Steps

Hopefully the ROC and precision-recall curves from the nested CV clued you in to the fact that we have somewhat resolved the overfitting of our random forest, although we may still want to consider other options. One issue to consider with regard to RF is that of **OVERTRAINING** - random forests are somewhat prone to it, despite all the great things about the algorithm. Secondly, we know this was a time-series of data, with different expression results more obvious at some time points than others (Verble et al. 2023). Thus, we would want to perform additional analyses before we feel too confident in these results. But we can see a positive impact of the nested CV.

3.1 Enter Phase 2 of this Project:

As we move forward with our benchmarking study, you have two choices. Choose your own adventure!

- **Path 1** - Potentially easier and more satisfying.

Perform k -fold & nested CV Random Forest, as well as hyperparameter tuning, on a different VST from the 30-minute experiment only.

If you decide to choose this adventure, you will use the VST data for the 30-min time point expression data only. You will also be given the DESeq results so you can use the `findOverlappingGenes()` function again to look at performance. Your task will be to take this vst, and work through all the same steps as in this first-half of the project. **Make sure to look at the important features at the end and comment on whether you have any confidence in your results!**

- **Path 2** - Potentially more challenging, and possibly dissatisfying.

Ready to do go farther? Try performing k -fold & nested CV, as well as hyperparameter tuning, on SVM.

If you choose this adventure, you will have the option to work through this same vst OR the 30-minute VST and your challenge is to apply SVM, including tuning the hyperparameters of **kernel**, **gamma**, and **cost**. Currently, we cannot get the important features out of nested cv SVM easily (alas - it needs another custom function!), so for this project you only need to engineer a best SVM model, discuss why you think it's the best model, and what your next recommendations are.

4 Final Comments / Advice

- Make sure to use the template I provide on GitHub for the project, rather than adding on to this document. It will just get too long! **If you can't upload the knitted file, just submit a note on Canvas that it wouldn't knit and that your RMD is on on your Team GitHub.* I expect some folks may have difficulty knitting. Be patient, too. It might take a half-hour or longer.
- **Don't forget!** Answer questions scattered throughout the Random Forest CV and Tuning Sections to ensure your understanding (you may copy these answers, without code, to your project document.)

Files you need for the 30-min data will be found in the `data_30min` directory:

1. `vst_30_min.Rdata` = the 30-min equivalent of the `vsData` object with the VST normalized read counts, formatted and ready to use as in this walk-through.
2. `vst_30_min.txt` = a text file version, which you likely do not need
3. `DESeq_on_30min_Data.Rmd` = an RMD file that was used to generate the 30-min data, plus some exploratory analysis. For the results see #4.
4. `DESeq_on_30_Data.html` = knitted file with some EDA and results to get you better acquainted to the data. Take a look! :)
5. `DESeqResults_30_min.Rdata` = the 30-min equivalent of the `DESeqResults` object that you can use with this walk-through.

You will absolutely need to take a look at #4 to get you started. You will also want to use #1 and #5 for your analysis.

References

- Love, Michael I., Wolfgang Huber, and Simon Anders. 2014. "Moderated Estimation of Fold Change and Dispersion for RNA-Seq Data with DESeq2." *Genome Biology* 15 (12): 550. <https://doi.org/10.1186/s13059-014-0550-8> (<https://doi.org/10.1186/s13059-014-0550-8>).
- R Core Team. 2019. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. <https://www.R-project.org> (<https://www.R-project.org>).
- Vabalas, Emma AND Poliakoff, Andrius AND Gowen. 2019. "Machine Learning Algorithm Validation with a Limited Sample Size." *PLOS ONE* 14 (11): 1–20. <https://doi.org/10.1371/journal.pone.0224365> (<https://doi.org/10.1371/journal.pone.0224365>).
- Verble, Kelton M., Ellen C. Keaveny, Sarthok Rasique Rahman, Matthew J. Jenny, Michael E. Dillon, and Jeffrey D. Lozier. 2024. "A Rapid Return to Normal: Temporal Gene Expression Patterns Following Cold Exposure in the Bumble Bee *Bombus Impatiens*." *Journal of Experimental Biology* 227 (9): jeb247040. <https://doi.org/10.1242/jeb.247040> (<https://doi.org/10.1242/jeb.247040>).